

UniVerse

A Campus Companion

Routing & Navigation

How the app decides which screen to show, and how every screen is reached.

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Fahmid (shared infrastructure)

Key files: `main.dart`, `core/router/*`, `shared/widgets/app_bottom_nav.dart`

1. The big picture (say this first)

UniVerse has **one** router for the whole app - a single GoRouter built in **app_router.dart**. There is no `Navigator.push(MaterialPageRoute(...))` anywhere. Every screen is reached through a **named route constant** (`RouteNames.*`), and a central **redirect** function decides, on every navigation, whether the user is allowed to be there.

- **route_names.dart** - every path string as a constant (no hardcoded '/login').
- **app_router.dart** - the GoRouter: the route table + the auth redirect guard.
- **app_shell.dart** - one Scaffold + bottom nav wrapped around all tab screens.
- **app_bottom_nav.dart** - the single source of truth for each role's tabs.

Say this: We use declarative routing - the URL is the single source of truth for what is on screen, and one guard protects every route instead of scattered checks.

2. Startup flow (main.dart)

- 1 **ensureInitialized()** + `AppTheme.setSystemUI()` - lock portrait, set status bar.
- 2 On mobile: **firebase.initializeApp()** then **PushService.init()** (skipped on web - there is no web Firebase config).
- 3 **Supabase.initialize()** with `AuthFlowType.pkce` - PKCE is the secure OAuth flow for mobile.
- 4 Create **AuthController** (the app's auth brain) and **AppRouter** (wraps GoRouter).
- 5 Subscribe to `authStateChanges`: on signed-in resolve the session + save the FCM token; on passwordRecovery jump to the reset screen.
- 6 **runApp(UniVerseApp)** -> `MaterialApp.router(theme: AppTheme.dark, routerConfig: router.router)`.

Important detail for the viva: the auth listener only resolves a session when no controller-driven flow is already doing it (email sign-in / OTP verify do it themselves). Double-resolving would race the router into the wrong redirect.

3. The redirect guard (the heart of routing)

`refreshListenable: authController` means **every time auth state changes, the router re-runs `redirect()`**. The guard reads `authController.status` and the target location, and returns either `null` (allow) or a path to bounce to.

auth status	what the guard does
initial / loading	Force to Splash until auth resolves.
notWhitelisted	Force to the Not-Whitelisted screen (rejected admin). Force to Verify-Email screen.
registering	Session exists but no profile yet (first Google login / OTP verified). Allow the auth/register pages; otherwise send to Role Selection.
unauthenticated	/ Allow auth pages; otherwise force to Login.
authenticated	If on an auth page -> send to the role's dashboard. If on the WRONG role's section (e.g. student on /admin) -> bounce to own dashboard.

`_dashboardForRole()` maps role -> landing route (student/teacher/admin dashboard).
`_isWrongRolePage()` enforces role isolation by URL prefix (/student, /teacher, /admin). This is why a student can never deep-link into an admin screen.

*Say this: The redirect is the **ONLY** place navigation rules live - screens never check 'is the user allowed here', they just exist; the guard decides.*

4. Route table: tabs vs secondary screens

The route list has two kinds of routes, and knowing the difference is a likely question:

a) Tab screens - inside the ShellRoute

A **ShellRoute** wraps a group of routes in one persistent **AppShell** (one Scaffold + one bottom nav). The child route only renders its content; the shell draws the nav bar around it. Tab screens: student dashboard/routine/resources/alerts/profile, teacher dashboard/routine/manage, admin dashboard/broadcast/routine/users.

b) Secondary screens - top-level routes (pushed)

Screens reached by a back button - NOT tabs - sit outside the shell so they have no bottom nav: Rooms, Find Teacher, Admin Registration, Manage Resources, Resource Library, Broadcast History, Manage Rooms/Faculty, Timetable Settings, Timetable Grid.

```
// passing data to a pushed route via `extra`:
context.push(RouteNames.timetableGrid, extra: rows);
// received:
TimetableGridScreen(rows: (s.extra as List<RoutineEntry>?) ?? const [])
```

5. AppShell - the persistent frame

app_shell.dart owns the only Scaffold for tab screens. It:

- Derives the active tab from the current router location (``GoRouterState.of(context).matchedLocation``) - screens don't track their own selection.
- Hosts the app-scoped **NotificationController**: starts ONE Realtime subscription for the whole session so the unread badge is live on every tab.
- Shows the **ExploreFabMenu** floating button only on the three dashboards (``/student|teacher|admin/dashboard``).
- Re-fetches notifications if the signed-in user changes, so one account's alerts never leak into another's badge.

6. AppBottomNav - role-aware tabs

``destinationsFor(role)`` is the **single source of truth** for tabs. A switch on role returns the list of ``AppNavDest(icon, label, route)``:

role	tabs
Student	Home - Routine - Resources - Alerts - Profile
Teacher	Home - Routine - Manage - Alerts - Profile
Admin	Dashboard - Broadcast - Routine - Users - Profile

The active index is computed by matching the current route; the Alerts tab shows the unread badge. Tapping calls ``context.go(route)`` (replaces, doesn't stack). Because tabs are derived from role + location, shared screens (Profile, Alerts) always show the correct tab set for whoever is logged in.

7. Likely defense questions

question	answer
Why GoRouter not Navigator?	Declarative + URL-driven; one central guard; deep links and role redirects are trivial. Navigator.push everywhere would scatter the auth checks.
How is a student blocked from /admin?	redirect() -> <code>_isWrongRolePage()</code> checks the URL prefix against the role and bounces to the user's own dashboard.
Why a ShellRoute?	So the bottom nav persists across tab switches (one Scaffold) instead of every screen rebuilding its own nav bar.
How do tabs stay in sync?	Active tab is derived from the router location, not stored - impossible to get out of sync.
Where are route strings?	All in <code>route_names.dart</code> - no screen hardcodes a path.